

Huffman Compression

Generated by Doxygen 1.13.2

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Compare	Comparator for priority queue ordering Huffman tree nodes	??
Node		??

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

headers/ huffman.h		??
headers/ utilities.h		??
src/ huffman.cpp		
Implements Huffman coding tree construction and decoding		??
src/ main.cpp		
Main entry point for the Huffman compression and decompression program		??
src/ utilities.cpp		
Utility functions for file handling, frequency calculations, and Huffman encoding support		??

Chapter 3

Class Documentation

3.1 Compare Struct Reference

Comparator for priority queue ordering Huffman tree nodes.

Public Member Functions

- bool [operator\(\)](#) ([Node](#) *left, [Node](#) *right)
Comparison operator for Huffman tree node priority.

3.1.1 Detailed Description

Comparator for priority queue ordering Huffman tree nodes.

Definition at line [17](#) of file [huffman.cpp](#).

3.1.2 Member Function Documentation

3.1.2.1 [operator\(\)](#)()

```
bool Compare::operator() (  
    Node * left,  
    Node * right) [inline]
```

Comparison operator for Huffman tree node priority.

Parameters

left	Pointer to left node.
right	Pointer to right node.

Returns

True if left node has higher frequency or same frequency but higher ASCII value.

Definition at line 25 of file [huffman.cpp](#).

```
00026 {
00027     if (left->frequency != right->frequency)
00028     {
00029         return left->frequency > right->frequency;
00030     }
00031     return left->character > right->character;
00032 }
```

The documentation for this struct was generated from the following file:

- [src/huffman.cpp](#)

3.2 Node Struct Reference

```
#include <huffman.h>
```

Public Member Functions

- [Node](#) (unsigned char c, int f)

Public Attributes

- unsigned char [character](#)
- int [frequency](#)
- [Node](#) * [left](#)
- [Node](#) * [right](#)

3.2.1 Detailed Description

Definition at line 9 of file [huffman.h](#).

3.2.2 Constructor & Destructor Documentation

3.2.2.1 Node()

```
Node::Node (
    unsigned char c,
    int f) [inline]
```

Definition at line 16 of file [huffman.h](#).

```
00016 : character(c), frequency(f), left(nullptr), right(nullptr) {}
```


3.2.3 Member Data Documentation

3.2.3.1 character

unsigned char Node::character

Definition at line 11 of file [huffman.h](#).

3.2.3.2 frequency

int Node::frequency

Definition at line 12 of file [huffman.h](#).

3.2.3.3 left

[Node*](#) Node::left

Definition at line 13 of file [huffman.h](#).

3.2.3.4 right

[Node*](#) Node::right

Definition at line 14 of file [huffman.h](#).

The documentation for this struct was generated from the following file:

- [headers/huffman.h](#)

Chapter 4

File Documentation

4.1 headers/huffman.h File Reference

```
#include <string>
#include <unordered_map>
#include <map>
```

Classes

- struct [Node](#)

Functions

- [Node](#) * [huffmanTree](#) (const std::map< unsigned char, int > &frequencyMap)
Builds the Huffman tree from a frequency map.
- void [printHuffmanTree](#) ([Node](#) *root, const std::string &code, std::map< unsigned char, std::string > &characters_codes)
Generates Huffman codes for characters by traversing the tree.
- void [printHuffmanTreeDebug](#) ([Node](#) *root, const std::string &prefix="")
Debugging function to print Huffman tree structure.
- std::string [decodeHuffmanCode](#) ([Node](#) *root, const std::string &encoded_text)
Decodes a Huffman-encoded bitstream.

4.1.1 Function Documentation

4.1.1.1 decodeHuffmanCode()

```
std::string decodeHuffmanCode (  
    Node * root,  
    const std::string & encoded_text)
```

Decodes a Huffman-encoded bitstream.

Parameters

root	Pointer to the root of the Huffman tree.
encoded_text	The encoded string of bits.

Returns

Decoded original text.

Definition at line 111 of file [huffman.cpp](#).

```

00112 {
00113     if (!root)
00114         return "";
00115     Node *current = root;
00116     std::string decoded_text = "";
00117     for (char bit : encoded_text)
00118     {
00119         if (bit == '0')
00120         {
00121             current = current->left;
00122         }
00123         else if (bit == '1')
00124         {
00125             current = current->right;
00126         }
00127         else
00128         {
00129             std::cerr << "Invalid bit in encoded text: " << bit << std::endl;
00130             return "";
00131         }
00132     }
00133     if (!current->left && !current->right)
00134     {
00135         decoded_text += current->character;
00136         current = root;
00137     }
00138 }
00139
00140 return decoded_text;
00141 }
00142
00143 }
```

4.1.1.2 huffmanTree()

Node * huffmanTree (
const std::map< unsigned char, int > & frequencyMap)

Builds the Huffman tree from a frequency map.

Parameters

frequencyMap	Map of character frequencies.
--------------	-------------------------------

Returns

Root node of the constructed Huffman tree.

Definition at line 40 of file [huffman.cpp](#).

```

00041 {
00042     std::priority_queue<Node *, std::vector<Node *>, Compare> heap;
00043     for (const auto &pair : frequencyMap)
00044     {
00045         heap.push(new Node(pair.first, pair.second));
00046     }
```

```

00047     }
00048
00049     while (heap.size() > 1)
00050     {
00051         Node *left = heap.top();
00052         heap.pop();
00053         Node *right = heap.top();
00054         heap.pop();
00055
00056         Node *internalNode = new Node('\0', left->frequency + right->frequency);
00057         internalNode->left = left;
00058         internalNode->right = right;
00059
00060         heap.push(internalNode);
00061     }
00062
00063     return heap.empty() ? nullptr : heap.top();
00064 }

```

4.1.1.3 printHuffmanTree()

```

void printHuffmanTree (
    Node * root,
    const std::string & code,
    std::map< unsigned char, std::string > & characters_codes)

```

Generates Huffman codes for characters by traversing the tree.

Parameters

root	Pointer to the root node of the Huffman tree.
code	Current binary string representation of the character.
characters_codes	Output map storing character-to-code mappings.

Definition at line 72 of file [huffman.cpp](#).

```

00073 {
00074     if (!root)
00075         return;
00076
00077     if (!root->left && !root->right)
00078     {
00079         characters_codes[root->character] = code;
00080     }
00081
00082     printHuffmanTree(root->left, code + "0", characters_codes);
00083     printHuffmanTree(root->right, code + "1", characters_codes);
00084 }

```

4.1.1.4 printHuffmanTreeDebug()

```

void printHuffmanTreeDebug (
    Node * root,
    const std::string & prefix)

```

Debugging function to print Huffman tree structure.

Parameters

root	Pointer to the root of the Huffman tree.
prefix	String representing the current tree path.

Definition at line 91 of file [huffman.cpp](#).

```
00092 {
00093     if (!root)
00094         return;
00095     if (!root->left && !root->right)
00096     {
00097         std::cout << "Leaf: " << root->character << " Code: " << prefix << std::endl;
00098     }
00099 }
00100
00101 printHuffmanTreeDebug(root->left, prefix + "0");
00102 printHuffmanTreeDebug(root->right, prefix + "1");
00103 }
```

4.2 huffman.h

[Go to the documentation of this file.](#)

```
00001 #ifndef HUFFMAN_H
00002 #define HUFFMAN_H
00003
00004 #include <string>
00005 #include <unordered_map>
00006 #include <map>
00007
00008 // NODE STRUCTURE
00009 struct Node
00010 {
00011     unsigned char character;
00012     int frequency;
00013     Node *left;
00014     Node *right;
00015
00016     Node(unsigned char c, int f) : character(c), frequency(f), left(nullptr), right(nullptr) {}
00017 };
00018
00019 Node *huffmanTree(const std::map<unsigned char, int> &frequencyMap);
00020 void printHuffmanTree(Node *root, const std::string &code, std::map<unsigned char, std::string> &characters_codes);
00021 void printHuffmanTreeDebug(Node *root, const std::string &prefix = "");
00022 std::string decodeHuffmanCode(Node *root, const std::string &encoded_text);
00023
00024 #endif
```

4.3 headers/utilities.h File Reference

```
#include <string>
#include <unordered_map>
#include <map>
```

Functions

- std::string [readFile](#) (const std::string &filename)
Reads an entire file into a string.
- void [writeFile](#) (const std::string &filename, const std::string &content)
Writes a string to a file.
- std::map< unsigned char, int > [calculateFrequencies](#) (const std::string &text)
Calculates the frequency of each character in a given text.
- std::map< unsigned char, int > [getDictionaryFrequencies](#) (const std::string &filename)
Reads character frequencies from a dictionary file.
- void [WriteCompressedFile](#) (const std::string &filename, std::string &bitstream)
Writes a compressed bitstream to a file with a magic header.
- void [writeCompressed](#) (const std::string &bitStream, const std::string &filenameWithoutExtension)
- std::string [readCompressed](#) (const std::string &filename)
Reads a compressed file and returns the extracted bitstream.

4.3.1 Function Documentation

4.3.1.1 calculateFrequencies()

```
std::map< unsigned char, int > calculateFrequencies (
    const std::string & text)
```

Calculates the frequency of each character in a given text.

Parameters

text	The input text.
------	-----------------

Returns

A map containing character frequencies.

Definition at line 45 of file [utilities.cpp](#).

```
00046 {
00047     std::map<unsigned char, int> frequencies;
00048     for (unsigned char ch : text)
00049     {
00050         frequencies[ch]++;
00051     }
00052     return frequencies;
00053 }
```

4.3.1.2 getDictionaryFrequencies()

```
std::map< unsigned char, int > getDictionaryFrequencies (
    const std::string & filename)
```

Reads character frequencies from a dictionary file.

Parameters

filename	The name of the dictionary file.
----------	----------------------------------

Returns

A map of character frequencies.

Definition at line 60 of file [utilities.cpp](#).

```
00061 {
00062     std::map<unsigned char, int> dictionary_frequencies;
00063     int charac;
00064     int freq;
00065     std::ifstream file(filename);
00066     if (!file.is_open())
00067     {
00068         std::cerr << "Error: Could not open dictionary file\n";
00069         return dictionary_frequencies;
00070     }
00071     while (file >> charac >> freq)
00072     {
00073         unsigned char character = static_cast<unsigned char>(charac);
00074         dictionary_frequencies[character] = freq;
00075     }
00076     file.close();
00077     return dictionary_frequencies;
00078 }
```

4.3.1.3 readCompressed()

```
std::string readCompressed (
    const std::string & filename)
```

Reads a compressed file and returns the extracted bitstream.

Parameters

filename	The name of the compressed file.
----------	----------------------------------

Returns

The extracted bitstream as a string.

Definition at line 122 of file [utilities.cpp](#).

```

00123 {
00124     std::ifstream in(filename, std::ios::binary);
00125     if (!in)
00126         throw std::runtime_error("Unable to open file for reading.");
00127
00128     uint32_t bitCount = 0;
00129     in.read(reinterpret_cast<char *>(&bitCount), sizeof(bitCount));
00130
00131     std::string bitStream;
00132     bitStream.reserve(bitCount);
00133
00134     unsigned char byte;
00135     while (in.read(reinterpret_cast<char *>(&byte), 1))
00136     {
00137         for (int i = 7; i >= 0; --i)
00138         {
00139             if (bitStream.size() < bitCount)
00140             {
00141                 bitStream.push_back((byte & (1 << i)) ? '1' : '0');
00142             }
00143             else
00144             {
00145                 break;
00146             }
00147         }
00148     }
00149     return bitStream;
00150 }
```

4.3.1.4 readFile()

```
std::string readFile (
    const std::string & filename)
```

Reads an entire file into a string.

Parameters

filename	The name of the file to read.
----------	-------------------------------

Returns

The contents of the file as a string.

Definition at line 21 of file [utilities.cpp](#).

```

00022 {
00023     std::ifstream file(filename);
00024     std::stringstream buffer;
00025     buffer << file.rdbuf();
00026     return buffer.str();
00027 }
```


4.3.1.5 writeCompressed()

```
void writeCompressed (
    const std::string & bitStream,
    const std::string & filenameWithoutExtension)
```

4.3.1.6 WriteCompressedFile()

```
void WriteCompressedFile (
    const std::string & filename,
    std::string & bitstream)
```

Writes a compressed bitstream to a file with a magic header.

Parameters

filename	The name of the output file.
bitstream	The compressed bitstream to write.

Definition at line 87 of file [utilities.cpp](#).

```
00088 {
00089     std::ofstream file(filename, std::ios::binary);
00090     if (!file.is_open())
00091     {
00092         std::cerr << "Error: Could not open the file for writing\n";
00093         return;
00094     }
00095     file.write(HUFF.c_str(), HUFF.size());
00096     char buffer = 0;
00097     int bitCount = 0;
00098     for (char bit : bitstream)
00099     {
00100         buffer = (buffer << 1) | (bit - '0');
00101         bitCount++;
00102         if (bitCount == 8)
00103         {
00104             file.put(buffer);
00105             buffer = 0;
00106             bitCount = 0;
00107         }
00108     }
00109     if (bitCount > 0)
00110     {
00111         buffer <<= (8 - bitCount);
00112         file.put(buffer);
00113     }
00114     file.close();
00115 }
```

4.3.1.7 writeFile()

```
void writeFile (
    const std::string & filename,
    const std::string & content)
```

Writes a string to a file.

Parameters

filename	The name of the file to write.
content	The content to write into the file.

Definition at line 34 of file [utilities.cpp](#).

```
00035 {
00036     std::ofstream file(filename, std::ios::out | std::ios::binary);
00037     file << content;
00038 }
```

4.4 utilities.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UTILITIES_H
00002 #define UTILITIES_H
00003
00004 #include <string>
00005 #include <unordered_map>
00006 #include <map>
00007
00008 // Function declarations
00009 std::string readFile(const std::string &filename);
00010 void writeFile(const std::string &filename, const std::string &content);
00011 std::map<unsigned char, int> calculateFrequencies(const std::string &text);
00012 std::map<unsigned char, int> getDictionaryFrequencies(const std::string &filename);
00013 void WriteCompressedFile(const std::string &filename, std::string &bitstream);
00014
00015 void writeCompressed(const std::string &bitStream, const std::string &filenameWithoutExtension);
00016 std::string readCompressed(const std::string &filename);
00017
00018 #endif
```

4.5 src/huffman.cpp File Reference

Implements Huffman coding tree construction and decoding.

```
#include "huffman.h"
#include <queue>
#include <iostream>
```

Classes

- struct [Compare](#)
Comparator for priority queue ordering Huffman tree nodes.

Functions

- [Node](#) * [huffmanTree](#) (const std::map< unsigned char, int > &frequencyMap)
Builds the Huffman tree from a frequency map.
- void [printHuffmanTree](#) ([Node](#) *root, const std::string &code, std::map< unsigned char, std::string > &characters_codes)
Generates Huffman codes for characters by traversing the tree.
- void [printHuffmanTreeDebug](#) ([Node](#) *root, const std::string &prefix)
Debugging function to print Huffman tree structure.
- std::string [decodeHuffmanCode](#) ([Node](#) *root, const std::string &encoded_text)
Decodes a Huffman-encoded bitstream.

4.5.1 Detailed Description

Implements Huffman coding tree construction and decoding.

This file contains the implementation of Huffman tree creation, encoding, and decoding functions for compressing and decompressing text.

Definition in file [huffman.cpp](#).

4.5.2 Function Documentation

4.5.2.1 decodeHuffmanCode()

```
std::string decodeHuffmanCode (
    Node * root,
    const std::string & encoded_text)
```

Decodes a Huffman-encoded bitstream.

Parameters

root	Pointer to the root of the Huffman tree.
encoded_text	The encoded string of bits.

Returns

Decoded original text.

Definition at line 111 of file [huffman.cpp](#).

```
00112 {
00113     if (!root)
00114         return "";
00115
00116     Node *current = root;
00117     std::string decoded_text = "";
00118
00119     for (char bit : encoded_text)
00120     {
00121         if (bit == '0')
00122         {
00123             current = current->left;
00124         }
00125         else if (bit == '1')
00126         {
00127             current = current->right;
00128         }
00129         else
00130         {
00131             std::cerr << "Invalid bit in encoded text: " << bit << std::endl;
00132             return "";
00133         }
00134
00135         if (!current->left && !current->right)
00136         {
00137             decoded_text += current->character;
00138             current = root;
00139         }
00140     }
00141
00142     return decoded_text;
00143 }
```

4.5.2.2 huffmanTree()

```
Node * huffmanTree (
    const std::map< unsigned char, int > & frequencyMap)
```

Builds the Huffman tree from a frequency map.

Parameters

frequencyMap	Map of character frequencies.
--------------	-------------------------------

Returns

Root node of the constructed Huffman tree.

Definition at line 40 of file [huffman.cpp](#).

```

00041 {
00042     std::priority_queue<Node *, std::vector<Node *>, Compare> heap;
00043     for (const auto &pair : frequencyMap)
00044     {
00045         heap.push(new Node(pair.first, pair.second));
00046     }
00047 }
00048
00049 while (heap.size() > 1)
00050 {
00051     Node *left = heap.top();
00052     heap.pop();
00053     Node *right = heap.top();
00054     heap.pop();
00055
00056     Node *internalNode = new Node('\0', left->frequency + right->frequency);
00057     internalNode->left = left;
00058     internalNode->right = right;
00059
00060     heap.push(internalNode);
00061 }
00062
00063 return heap.empty() ? nullptr : heap.top();
00064 }
```

4.5.2.3 printHuffmanTree()

```

void printHuffmanTree (
    Node * root,
    const std::string & code,
    std::map< unsigned char, std::string > & characters_codes)
```

Generates Huffman codes for characters by traversing the tree.

Parameters

root	Pointer to the root node of the Huffman tree.
code	Current binary string representation of the character.
characters_codes	Output map storing character-to-code mappings.

Definition at line 72 of file [huffman.cpp](#).

```

00073 {
00074     if (!root)
00075         return;
00076
00077     if (!root->left && !root->right)
00078     {
00079         characters_codes[root->character] = code;
00080     }
00081
00082     printHuffmanTree(root->left, code + "0", characters_codes);
00083     printHuffmanTree(root->right, code + "1", characters_codes);
00084 }
```

4.5.2.4 printHuffmanTreeDebug()

```

void printHuffmanTreeDebug (
    Node * root,
    const std::string & prefix)
```

Debugging function to print Huffman tree structure.

Parameters

root	Pointer to the root of the Huffman tree.
prefix	String representing the current tree path.

Definition at line 91 of file [huffman.cpp](#).

```

00092 {
00093     if (!root)
00094         return;
00095
00096     if (!root->left && !root->right)
00097     {
00098         std::cout << "Leaf: " << root->character << " Code: " << prefix << std::endl;
00099     }
00100
00101     printHuffmanTreeDebug(root->left, prefix + "0");
00102     printHuffmanTreeDebug(root->right, prefix + "1");
00103 }
```

4.6 huffman.cpp

[Go to the documentation of this file.](#)

```

00001
00008
00009 #include "huffman.h"
00010 #include <queue>
00011 #include <iostream>
00012
00017 struct Compare
00018 {
00025     bool operator()(Node *left, Node *right)
00026     {
00027         if (left->frequency != right->frequency)
00028         {
00029             return left->frequency > right->frequency;
00030         }
00031         return left->character > right->character;
00032     }
00033 };
00034
00040 Node *huffmanTree(const std::map<unsigned char, int> &frequencyMap)
00041 {
00042     std::priority_queue<Node *, std::vector<Node *>, Compare> heap;
00043
00044     for (const auto &pair : frequencyMap)
00045     {
00046         heap.push(new Node(pair.first, pair.second));
00047     }
00048
00049     while (heap.size() > 1)
00050     {
00051         Node *left = heap.top();
00052         heap.pop();
00053         Node *right = heap.top();
00054         heap.pop();
00055
00056         Node *internalNode = new Node('\0', left->frequency + right->frequency);
00057         internalNode->left = left;
00058         internalNode->right = right;
00059
00060         heap.push(internalNode);
00061     }
00062
00063     return heap.empty() ? nullptr : heap.top();
00064 }
00065
00072 void printHuffmanTree(Node *root, const std::string &code, std::map<unsigned char, std::string> &characters_codes)
00073 {
00074     if (!root)
00075         return;
00076
00077     if (!root->left && !root->right)
00078     {
00079         characters_codes[root->character] = code;
00080     }
00081 }
```

```

00082     printHuffmanTree(root->left, code + "0", characters_codes);
00083     printHuffmanTree(root->right, code + "1", characters_codes);
00084 }
00085
00091 void printHuffmanTreeDebug(Node *root, const std::string &prefix)
00092 {
00093     if (!root)
00094         return;
00095
00096     if (!root->left && !root->right)
00097     {
00098         std::cout << "Leaf: " << root->character << " Code: " << prefix << std::endl;
00099     }
00100
00101     printHuffmanTreeDebug(root->left, prefix + "0");
00102     printHuffmanTreeDebug(root->right, prefix + "1");
00103 }
00104
00111 std::string decodeHuffmanCode(Node *root, const std::string &encoded_text)
00112 {
00113     if (!root)
00114         return "";
00115
00116     Node *current = root;
00117     std::string decoded_text = "";
00118
00119     for (char bit : encoded_text)
00120     {
00121         if (bit == '0')
00122         {
00123             current = current->left;
00124         }
00125         else if (bit == '1')
00126         {
00127             current = current->right;
00128         }
00129         else
00130         {
00131             std::cerr << "Invalid bit in encoded text: " << bit << std::endl;
00132             return "";
00133         }
00134
00135         if (!current->left && !current->right)
00136         {
00137             decoded_text += current->character;
00138             current = root;
00139         }
00140     }
00141
00142     return decoded_text;
00143 }

```

4.7 src/main.cpp File Reference

Main entry point for the Huffman compression and decompression program.

```

#include <iostream>
#include <iomanip>
#include <map>
#include "huffman.h"
#include "utilities.h"

```

Functions

- void [printHelp](#) ()
Prints help information for using the program.
- int [main](#) (int argc, char *argv[])
Main function handling command-line arguments and program execution.

4.7.1 Detailed Description

Main entry point for the Huffman compression and decompression program.

This file handles command-line arguments, file I/O, and calls the Huffman encoding and decoding functions accordingly.

Definition in file [main.cpp](#).

4.7.2 Function Documentation

4.7.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Main function handling command-line arguments and program execution.

Parameters

argc	Argument count.
argv	Argument vector.

Returns

0 on success, 1 on failure.

Definition at line 37 of file [main.cpp](#).

```
00038 {
00039     #ifdef _WIN32
00040         SetConsoleOutputCP(CP_UTF8);
00041     #endif
00042
00043     if (argc == 1)
00044     {
00045         printHelp();
00046         return 0;
00047     }
00048
00049     // Command-line argument parsing
00050     std::string inputFile, outputFile, dictionaryFile, mode;
00051     for (int i = 1; i < argc; i += 2)
00052     {
00053         std::string flag = argv[i];
00054         if (i + 1 >= argc)
00055         {
00056             std::cerr << "Error: missing value for " << flag << "\n";
00057             return 1;
00058         }
00059         if (flag == "-i")
00060         {
00061             inputFile = argv[i + 1];
00062         }
00063         else if (flag == "-o")
00064         {
00065             outputFile = argv[i + 1];
00066         }
00067         else if (flag == "-d")
00068         {
00069             dictionaryFile = argv[i + 1];
00070         }
00071         else if (flag == "-m")
00072         {
00073             mode = argv[i + 1];
```

```

00074     }
00075     else
00076     {
00077         std::cerr << "Error: Unknown flag " << flag << "\n";
00078         return 1;
00079     }
00080 }
00081
00082 // Validate arguments
00083 if (inputFile.empty() || outputFile.empty() || mode.empty())
00084 {
00085     std::cerr << "Error: Missing required arguments\n";
00086     printHelp();
00087     return 1;
00088 }
00089
00090 // Compression Mode
00091 if (mode == "c")
00092 {
00093     std::string buffer = readFile(inputFile);
00094     auto characters_frequency = calculateFrequencies(buffer);
00095     Node *root = huffmanTree(characters_frequency);
00096     std::map<unsigned char, std::string> characters_codes;
00097     printHuffmanTree(root, "", characters_codes);
00098
00099     std::string encoded_text;
00100     for (unsigned char ch : buffer)
00101     {
00102         encoded_text += characters_codes[ch];
00103     }
00104     WriteCompressedFile(outputFile, encoded_text);
00105 }
00106 // Decompression Mode
00107 else if (mode == "d")
00108 {
00109     std::map<unsigned char, int> dictionary_frequencies = getDictionaryFrequencies(dictionaryFile);
00110     Node *root = huffmanTree(dictionary_frequencies);
00111     std::string DECODING = readCompressed(inputFile);
00112     std::string decoded_text = decodeHuffmanCode(root, DECODING);
00113     writeFile(outputFile, decoded_text);
00114 }
00115 else
00116 {
00117     std::cerr << "Error: Invalid mode. Use 'c' for compress or 'd' for decompress.\n";
00118     printHelp();
00119     return 1;
00120 }
00121 }

```

4.7.2.2 printHelp()

void printHelp ()

Prints help information for using the program.

Definition at line 23 of file [main.cpp](#).

```

00024 {
00025     std::cout << "\nUsage: ./huffman -i <input_file> -o <output_file> -m <mode> -d <dictionary_file>\n";
00026     std::cout << "Modes:\n  c : compress\n  d : decompress\n";
00027     std::cout << "Example:\n  ./huffman -i input.txt -o compressed.huff -m c -d Dictionary.txt\n";
00028     std::cout << "Please REMEMBER that the compressed file must have the .huff extension!\n";
00029 }

```

4.8 main.cpp

[Go to the documentation of this file.](#)

```

00001
00008
00009 #include <iostream>
00010 #include <iomanip>
00011 #include <map>
00012
00013 #include "huffman.h"
00014 #include "utilities.h"

```



```

00015
00016 #ifdef _WIN32
00017 #include <windows.h>
00018 #endif
00019
00023 void printHelp()
00024 {
00025     std::cout << "\nUsage: ./huffman -i <input_file> -o <output_file> -m <mode> -d <dictionary_file>\n";
00026     std::cout << "Modes:\n  c : compress\n  d : decompress\n";
00027     std::cout << "Example:\n  ./huffman -i input.txt -o compressed.huff -m c -d Dictionary.txt\n";
00028     std::cout << "Please REMEMBER that the compressed file must have the .huff extension!\n";
00029 }
00030
00037 int main(int argc, char *argv[])
00038 {
00039     #ifdef _WIN32
00040         SetConsoleOutputCP(CP_UTF8);
00041     #endif
00042
00043     if (argc == 1)
00044     {
00045         printHelp();
00046         return 0;
00047     }
00048
00049     // Command-line argument parsing
00050     std::string inputFile, outputFile, dictionaryFile, mode;
00051     for (int i = 1; i < argc; i += 2)
00052     {
00053         std::string flag = argv[i];
00054         if (i + 1 >= argc)
00055         {
00056             std::cerr << "Error: missing value for " << flag << "\n";
00057             return 1;
00058         }
00059         if (flag == "-i")
00060         {
00061             inputFile = argv[i + 1];
00062         }
00063         else if (flag == "-o")
00064         {
00065             outputFile = argv[i + 1];
00066         }
00067         else if (flag == "-d")
00068         {
00069             dictionaryFile = argv[i + 1];
00070         }
00071         else if (flag == "-m")
00072         {
00073             mode = argv[i + 1];
00074         }
00075         else
00076         {
00077             std::cerr << "Error: Unknown flag " << flag << "\n";
00078             return 1;
00079         }
00080     }
00081
00082     // Validate arguments
00083     if (inputFile.empty() || outputFile.empty() || mode.empty())
00084     {
00085         std::cerr << "Error: Missing required arguments\n";
00086         printHelp();
00087         return 1;
00088     }
00089
00090     // Compression Mode
00091     if (mode == "c")
00092     {
00093         std::string buffer = readFile(inputFile);
00094         auto characters_frequency = calculateFrequencies(buffer);
00095         Node *root = huffmanTree(characters_frequency);
00096         std::map<unsigned char, std::string> characters_codes;
00097         printHuffmanTree(root, "", characters_codes);
00098
00099         std::string encoded_text;
00100         for (unsigned char ch : buffer)
00101         {
00102             encoded_text += characters_codes[ch];
00103         }
00104         WriteCompressedFile(outputFile, encoded_text);
00105     }
00106     // Decompression Mode
00107     else if (mode == "d")
00108     {
00109         std::map<unsigned char, int> dictionary_frequencies = getDictionaryFrequencies(dictionaryFile);
00110         Node *root = huffmanTree(dictionary_frequencies);

```

```

00111     std::string DECODING = readCompressed(inputFile);
00112     std::string decoded_text = decodeHuffmanCode(root, DECODING);
00113     writeFile(outputFile, decoded_text);
00114 }
00115 else
00116 {
00117     std::cerr << "Error: Invalid mode. Use 'c' for compress or 'd' for decompress.\n";
00118     printHelp();
00119     return 1;
00120 }
00121 }

```

4.9 src/utilities.cpp File Reference

Utility functions for file handling, frequency calculations, and Huffman encoding support.

```

#include "utilities.h"
#include <fstream>
#include <sstream>
#include <iostream>
#include <map>
#include <string>
#include <vector>
#include <stdexcept>
#include <cstdlib>

```

Functions

- `std::string readFile (const std::string &filename)`
Reads an entire file into a string.
- `void writeFile (const std::string &filename, const std::string &content)`
Writes a string to a file.
- `std::map< unsigned char, int > calculateFrequencies (const std::string &text)`
Calculates the frequency of each character in a given text.
- `std::map< unsigned char, int > getDictionaryFrequencies (const std::string &filename)`
Reads character frequencies from a dictionary file.
- `void WriteCompressedFile (const std::string &filename, std::string &bitstream)`
Writes a compressed bitstream to a file with a magic header.
- `std::string readCompressed (const std::string &filename)`
Reads a compressed file and returns the extracted bitstream.

Variables

- `const std::string HUFF = "HUFF"`

4.9.1 Detailed Description

Utility functions for file handling, frequency calculations, and Huffman encoding support.

Definition in file [utilities.cpp](#).

4.9.2 Function Documentation

4.9.2.1 calculateFrequencies()

```

std::map< unsigned char, int > calculateFrequencies (
    const std::string & text)

```

Calculates the frequency of each character in a given text.

Parameters

text	The input text.
------	-----------------

Returns

A map containing character frequencies.

Definition at line 45 of file [utilities.cpp](#).

```
00046 {
00047     std::map<unsigned char, int> frequencies;
00048     for (unsigned char ch : text)
00049     {
00050         frequencies[ch]++;
00051     }
00052     return frequencies;
00053 }
```

4.9.2.2 getDictionaryFrequencies()

```
std::map< unsigned char, int > getDictionaryFrequencies (
    const std::string & filename)
```

Reads character frequencies from a dictionary file.

Parameters

filename	The name of the dictionary file.
----------	----------------------------------

Returns

A map of character frequencies.

Definition at line 60 of file [utilities.cpp](#).

```
00061 {
00062     std::map<unsigned char, int> dictionary_frequencies;
00063     int charac;
00064     int freq;
00065     std::ifstream file(filename);
00066     if (!file.is_open())
00067     {
00068         std::cerr << "Error: Could not open dictionary file\n";
00069         return dictionary_frequencies;
00070     }
00071     while (file >> charac >> freq)
00072     {
00073         unsigned char character = static_cast<unsigned char>(charac);
00074         dictionary_frequencies[character] = freq;
00075     }
00076     file.close();
00077     return dictionary_frequencies;
00078 }
```

4.9.2.3 readCompressed()

```
std::string readCompressed (
    const std::string & filename)
```

Reads a compressed file and returns the extracted bitstream.

Parameters

filename	The name of the compressed file.
----------	----------------------------------

Returns

The extracted bitstream as a string.

Definition at line 122 of file [utilities.cpp](#).

```

00123 {
00124     std::ifstream in(filename, std::ios::binary);
00125     if (!in)
00126         throw std::runtime_error("Unable to open file for reading.");
00127
00128     uint32_t bitCount = 0;
00129     in.read(reinterpret_cast<char *>(&bitCount), sizeof(bitCount));
00130
00131     std::string bitStream;
00132     bitStream.reserve(bitCount);
00133
00134     unsigned char byte;
00135     while (in.read(reinterpret_cast<char *>(&byte), 1))
00136     {
00137         for (int i = 7; i >= 0; --i)
00138         {
00139             if (bitStream.size() < bitCount)
00140             {
00141                 bitStream.push_back((byte & (1 << i)) ? '1' : '0');
00142             }
00143             else
00144             {
00145                 break;
00146             }
00147         }
00148     }
00149     return bitStream;
00150 }
```

4.9.2.4 readFile()

```
std::string readFile (
    const std::string & filename)
```

Reads an entire file into a string.

Parameters

filename	The name of the file to read.
----------	-------------------------------

Returns

The contents of the file as a string.

Definition at line 21 of file [utilities.cpp](#).

```

00022 {
00023     std::ifstream file(filename);
00024     std::stringstream buffer;
00025     buffer << file.rdbuf();
00026     return buffer.str();
00027 }
```

4.9.2.5 WriteCompressedFile()

```
void WriteCompressedFile (
    const std::string & filename,
    std::string & bitstream)
```

Writes a compressed bitstream to a file with a magic header.

Parameters

filename	The name of the output file.
bitstream	The compressed bitstream to write.

Definition at line 87 of file [utilities.cpp](#).

```

00088 {
00089     std::ofstream file(filename, std::ios::binary);
00090     if (!file.is_open())
00091     {
00092         std::cerr << "Error: Could not open the file for writing\n";
00093         return;
00094     }
00095     file.write(HUFF.c_str(), HUFF.size());
00096     char buffer = 0;
00097     int bitCount = 0;
00098     for (char bit : bitstream)
00099     {
00100         buffer = (buffer << 1) | (bit - '0');
00101         bitCount++;
00102         if (bitCount == 8)
00103         {
00104             file.put(buffer);
00105             buffer = 0;
00106             bitCount = 0;
00107         }
00108     }
00109     if (bitCount > 0)
00110     {
00111         buffer <<= (8 - bitCount);
00112         file.put(buffer);
00113     }
00114     file.close();
00115 }
```

4.9.2.6 writeFile()

```

void writeFile (
    const std::string & filename,
    const std::string & content)
```

Writes a string to a file.

Parameters

filename	The name of the file to write.
content	The content to write into the file.

Definition at line 34 of file [utilities.cpp](#).

```

00035 {
00036     std::ofstream file(filename, std::ios::out | std::ios::binary);
00037     file << content;
00038 }
```

4.9.3 Variable Documentation

4.9.3.1 HUFF

```
const std::string HUFF = "HUFF"
```

Definition at line 80 of file [utilities.cpp](#).

4.10 utilities.cpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #include "utilities.h"
00007 #include <fstream>
00008 #include <sstream>
00009 #include <iostream>
00010 #include <map>
00011 #include <string>
00012 #include <vector>
00013 #include <stdexcept>
00014 #include <cstdint>
00015
00021 std::string readFile(const std::string &filename)
00022 {
00023     std::ifstream file(filename);
00024     std::stringstream buffer;
00025     buffer << file.rdbuf();
00026     return buffer.str();
00027 }
00028
00034 void writeFile(const std::string &filename, const std::string &content)
00035 {
00036     std::ofstream file(filename, std::ios::out | std::ios::binary);
00037     file << content;
00038 }
00039
00045 std::map<unsigned char, int> calculateFrequencies(const std::string &text)
00046 {
00047     std::map<unsigned char, int> frequencies;
00048     for (unsigned char ch : text)
00049     {
00050         frequencies[ch]++;
00051     }
00052     return frequencies;
00053 }
00054
00060 std::map<unsigned char, int> getDictionaryFrequencies(const std::string &filename)
00061 {
00062     std::map<unsigned char, int> dictionary_frequencies;
00063     int charac;
00064     int freq;
00065     std::ifstream file(filename);
00066     if (!file.is_open())
00067     {
00068         std::cerr << "Error: Could not open dictionary file\n";
00069         return dictionary_frequencies;
00070     }
00071     while (file >> charac >> freq)
00072     {
00073         unsigned char character = static_cast<unsigned char>(charac);
00074         dictionary_frequencies[character] = freq;
00075     }
00076     file.close();
00077     return dictionary_frequencies;
00078 }
00079
00080 const std::string HUFF = "HUFF";
00081
00087 void WriteCompressedFile(const std::string &filename, std::string &bitstream)
00088 {
00089     std::ofstream file(filename, std::ios::binary);
00090     if (!file.is_open())
00091     {
00092         std::cerr << "Error: Could not open the file for writing\n";
00093         return;
00094     }
00095     file.write(HUFF.c_str(), HUFF.size());
00096     char buffer = 0;
00097     int bitCount = 0;
00098     for (char bit : bitstream)
00099     {
00100         buffer = (buffer << 1) | (bit - '0');
00101         bitCount++;
00102         if (bitCount == 8)
00103         {
00104             file.put(buffer);
00105             buffer = 0;
00106             bitCount = 0;
00107         }
00108     }
00109     if (bitCount > 0)
00110     {

```

```
00111     buffer «= (8 - bitCount);
00112     file.put(buffer);
00113 }
00114 file.close();
00115 }
00116
00122 std::string readCompressed(const std::string &filename)
00123 {
00124     std::ifstream in(filename, std::ios::binary);
00125     if (!in)
00126         throw std::runtime_error("Unable to open file for reading.");
00127
00128     uint32_t bitCount = 0;
00129     in.read(reinterpret_cast<char *>(&bitCount), sizeof(bitCount));
00130
00131     std::string bitStream;
00132     bitStream.reserve(bitCount);
00133
00134     unsigned char byte;
00135     while (in.read(reinterpret_cast<char *>(&byte), 1))
00136     {
00137         for (int i = 7; i >= 0; --i)
00138         {
00139             if (bitStream.size() < bitCount)
00140             {
00141                 bitStream.push_back((byte & (1 « i)) ? '1' : '0');
00142             }
00143             else
00144             {
00145                 break;
00146             }
00147         }
00148     }
00149     return bitStream;
00150 }
```

